

Timothy Song

Professor Shim

DATA 260

28 September 2026

Homework 4 Report

GitHub Repository: <https://github.com/TimothyYSong/data260-9981>

Section 0 - Personal Configuration and Domain

Hardware: MacBook Pro M4 Pro, 14 cores (10 performance and 4 efficiency), 24 GB memory

Models: LLM: qwen3:8b and Embedding model: nomic-embed-text

Tagged commit hash:

SID4: 9981

PORT_BASE: 8081

PREFIX: s9981

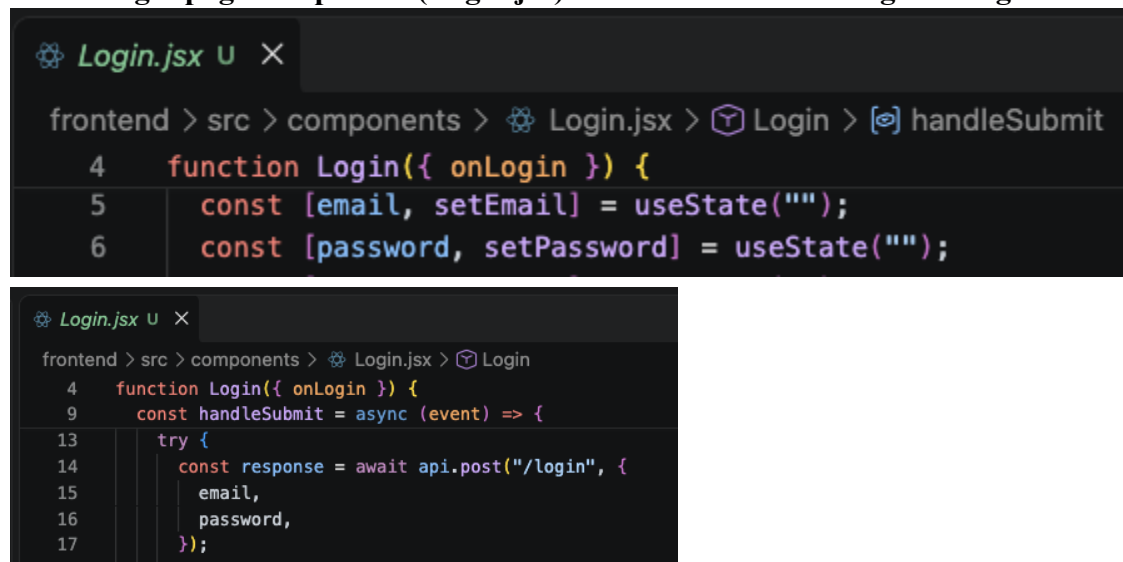
SEED: 9981

VERIFY_SEED: 269981

DOMAIN_ID: 5 (Local restaurant inspections)

Part 1 - React Client

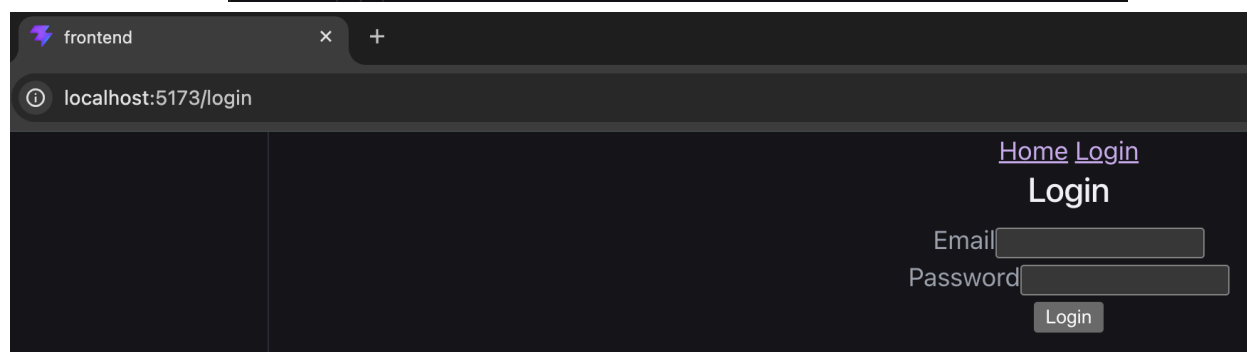
Add a Login page/component (Login.jsx) that allows a user to log in using email + password.



```
frontend > src > components > Login.jsx > Login > handleSubmit
4  function Login({ onLogin }) {
5      const [email, setEmail] = useState("");
6      const [password, setPassword] = useState("");

13
14      const response = await api.post("/login", {
15          email,
16          password,
17      });
```

```
Login.jsx U X
frontend > src > components > Login.jsx > default
4  function Login({ onLogin }) {
27  return (
28    <div>
29      <h2>Login</h2>
30
31      {error && <p>{error}</p>}
32
33      <form onSubmit={handleSubmit}>
34        <div>
35          <label htmlFor="email">Email</label>
36          <input
37            id="email"
38            type="email"
39            value={email}
40            onChange={(event) => setEmail(event.target.value)}
41            required
42          />
43        </div>
44
45        <div>
46          <label htmlFor="password">Password</label>
47          <input
48            id="password"
49            type="password"
50            value={password}
51            onChange={(event) => setPassword(event.target.value)}
52            required
53          />
54        </div>
55
56        <button type="submit">Login</button>
```



After successful login, the backend will set an HTTP-only cookie containing a server-side session token.

```
auth.py M X
code > routers > auth.py > login
21 def login(
48
49     session_token = secrets.token_urlsafe(32)
50
```

```
auth.py M X
code > routers > auth.py > get_current_user
21 def login(
65
66     response.set_cookie(
67         key=SESSION_COOKIE_NAME,
68         value=session_token,
69         httponly=True,
70         samesite="lax",
71         secure=False,
72     )
```

```
auth.py M X
code > routers > auth.py > login
21 def login(
56     session = SessionModel(
57         id=session_token,
58         user_id=user.id,
59         created_at=created_at,
60         expires_at=expires_at,
61     )
62
63     db.add(session)
64     db.commit()
65
```

frontend

localhost:5173/login

Application

Manifest

Service workers

Storage

WebMCP NEW

Ads

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

http://localhost:5173

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition k...	Cross Site	Priority
session_token	2FWdNV1bDqe6WpVKHU3_dJ4_LLCRDhj5T4m84ciQU	localhost	/	Session	56	✓		Lax			Medium

Only logged-in users should be allowed to view the record list and perform CRUD operations. If the user is not logged in: show a message "Login required", and/or disable/hide navigation buttons like "Add Record".

```

Home.jsx
frontend > src > pages > Home.jsx > Home
4  function Home({ user }) {
25    if (!user) {
26      return (
27        <div>
28          <h1>Local Restaurant Inspections</h1>
29          <p>Login required</p>
30        </div>
31      );
32    }

```

```

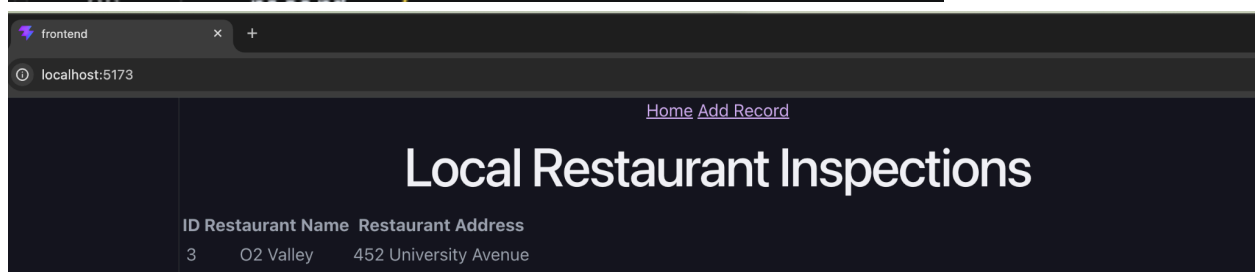
CreateRecord.jsx
frontend > src > pages > CreateRecord.jsx > CreateRecord > handleSubmit
4  function CreateRecord({ user, addRecord }) {
10    if (!user) {
11      return <p>Login required</p>;
12    }

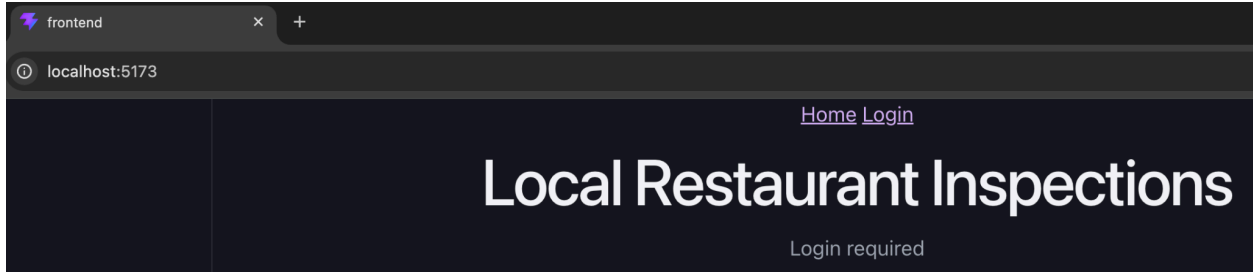
```

```

records.py
code > routers > records.py > get_record
92  @router.delete("/{record_id}")
93  def delete_record(
94      record_id: int,
95      db: DatabaseSession = Depends(get_db),
96      user: User = Depends(get_current_user),
97  ):

```



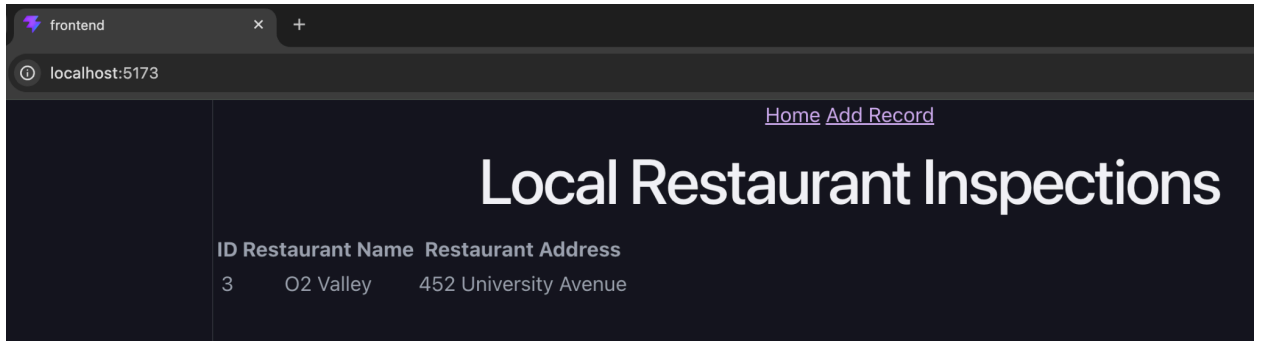


I. Home Page

```
Home.jsx U X
frontend > src > pages > Home.jsx > default
4  function Home({ user }) {
5    const [records, setRecords] = useState([]);
6    const [error, setError] = useState("");
7
8    useEffect(() => {
9      if (!user) {
10       return;
11     }
12
13     const fetchRecords = async () => {
14       try {
15         const response = await api.get("/records");
16         setRecords(response.data);
17       } catch {
18         setError("Unable to load records.");
19       }
20     };
21
22     fetchRecords();
23   }, [user]);
```

```
Home.jsx U X
frontend > src > pages > Home.jsx > default
4  function Home({ user }) {
52    <tbody>
53      {records.map((record) => (
54        <tr key={record.id}>
55          <td>{record.id}</td>
56          <td>{record.restaurant_name}</td>
57          <td>{record.restaurant_address}</td>
58        </tr>
59      )]}
60    </tbody>
```

```
App.jsx U X
frontend > src > App.jsx > default
9  function App() {
22    <Route path="/" element={<Home user={user} />} />
```



II. Add a New Record

```

CreateRecord.jsx U X
frontend > src > pages > CreateRecord.jsx > default
1  import { useState } from "react";
2  import { useNavigate } from "react-router-dom";
3
4  function CreateRecord({ user, addRecord }) {
5    const [restaurantName, setRestaurantName] = useState("");
6    const [restaurantAddress, setRestaurantAddress] = useState("");
7    const [error, setError] = useState("");
8    const navigate = useNavigate();
9
10   if (!user) {
11     return <p>Login required</p>;
12   }
13
14   const handleSubmit = async (event) => {
15     event.preventDefault();
16     setError("");
17
18     try {
19       await addRecord({
20         restaurant_name: restaurantName,
21         restaurant_address: restaurantAddress,
22       });
23
24       navigate("/");
25     } catch {
26       setError("Unable to create record.");
27     }
28   };
29
30   return (
31     <div>
32       <h1>Add Restaurant Inspection</h1>
33
34       {error && <p>{error}</p>}
35
36       <form onSubmit={handleSubmit}>
37         <div>
38           <label htmlFor="restaurantName">Restaurant Name</label>
39           <input
40             id="restaurantName"
41             type="text"
42             value={restaurantName}
43             onChange={(event) => setRestaurantName(event.target.value)}
44             required
45           />
46         </div>
47
48         <div>
49           <label htmlFor="restaurantAddress">Restaurant Address</label>
50           <input
51             id="restaurantAddress"
52             type="text"
53             value={restaurantAddress}
54             onChange={(event) => setRestaurantAddress(event.target.value)}
55             required
56           />
57         </div>
58
59         <button type="submit">Add Inspection</button>
60       </form>
61     </div>
62   );
63 }
64
65 export default CreateRecord;

```

```
App.jsx U X
frontend > src > App.jsx > [🔍] default
10  function App() {
12      const addRecord = async (record) => {
13          await api.post("/records", record);
14      };
```

```
App.jsx U X
frontend > src > App.jsx > [🔍] default
10  function App() {
28      <Route path="/create" element={<CreateRecord user={user} addRecord={addRecord} />} />
```

The top screenshot shows a web browser at localhost:5173/create. The page has a dark theme and a sidebar. The main content area has a header with links 'Home' and 'Add Record'. Below the header is a large heading 'Add Restaurant Inspection'. There are two input fields: 'Restaurant Name' with the value 'Pho Nam' and 'Restaurant Address' with the value '911 W El Camino Real'. Below these fields is a button labeled 'Add Inspection'.

The bottom screenshot shows the same web browser at localhost:5173. The page has a dark theme and a sidebar. The main content area has a header with links 'Home' and 'Add Record'. Below the header is a large heading 'Local Restaurant Inspections'. Below the heading is a table with three columns: 'ID', 'Restaurant Name', and 'Restaurant Address'. The table contains two rows of data:

ID	Restaurant Name	Restaurant Address
3	O2 Valley	452 University Avenue
4	Pho Nam	911 W El Camino Real

III. Update Record

```
App.jsx U X
frontend > src > App.jsx > default
10  function App() {
32  }
    <Route path="/update" element={ <UpdateRecord user={user} updateRecord={updateRecord} /> } />
```

```
App.jsx U X
frontend > src > App.jsx > default
10  function App() {
15      const updateRecord = async (id, record) => {
16          await api.put(`/records/${id}`, record);
17      };
    };
```

```

UpdateRecord.jsx U X
frontend > src > pages > UpdateRecord.jsx > default
4 function UpdateRecord({ user, updateRecord }) {
15   const handleSubmit = async (event) => {
16     event.preventDefault();
17     setError("");
18
19     try {
20       await updateRecord(recordId, {
21         restaurant_name: restaurantName,
22         restaurant_address: restaurantAddress,
23       });
24
25       navigate("/");
26     } catch {
27       setError("Unable to update record.");
28     }
29   };
30
31   return (
32     <div>
33       <h1>Update Restaurant Inspection</h1>
34
35       {error && <p>{error}</p>}
36
37       <form onSubmit={handleSubmit}>
38         <div>
39           <label htmlFor="recordId">Record ID</label>
40           <input
41             id="recordId"
42             type="number"
43             value={recordId}
44             onChange={(event) => setRecordId(event.target.value)}
45             required
46           />
47         </div>
48
49         <div>
50           <label htmlFor="restaurantName">Restaurant Name</label>
51           <input
52             id="restaurantName"
53             type="text"
54             value={restaurantName}
55             onChange={(event) => setRestaurantName(event.target.value)}
56             required
57           />
58         </div>
59
60         <div>
61           <label htmlFor="restaurantAddress">Restaurant Address</label>
62           <input
63             id="restaurantAddress"
64             type="text"
65             value={restaurantAddress}
66             onChange={(event) => setRestaurantAddress(event.target.value)}
67             required
68           />
69         </div>
70
71         <button type="submit">Update Inspection</button>
72       </form>
73     </div>
74   );
75 }
76
77 export default UpdateRecord;

```

The image shows two screenshots of a web application. The top screenshot displays the 'Update Restaurant Inspection' form, and the bottom screenshot displays the 'Local Restaurant Inspections' table.

Update Restaurant Inspection Form:

- Record ID: 3
- Restaurant Name: O2 Valley Palo Alto
- Restaurant Address: 452 University Avenue f
- Update Inspection button

Local Restaurant Inspections Table:

ID	Restaurant Name	Restaurant Address
3	O2 Valley Palo Alto	452 University Avenue Palo Alto
4	Pho Nam	911 W El Camino Real

IV. Delete the Record

```
App.jsx U X
frontend > src > App.jsx > default
10  function App() {
38  <Route path="/delete" element={}<DeleteRecord user={user} deleteRecord={deleteRecord} /> } />
39  />
```

```
App.jsx U X
frontend > src > App.jsx > default
10  function App() {
19
20      const deleteRecord = async (id) => {
21          await api.delete(`/records/${id}`);
22      };
23  }
```

```
DeleteRecord.jsx U X
frontend > src > pages > DeleteRecord.jsx > default
3
4 function DeleteRecord({ user, deleteRecord }) {
5   const [recordId, setRecordId] = useState("");
6   const [error, setError] = useState("");
7   const navigate = useNavigate();
8
9   if (!user) {
10    return <p>Login required</p>;
11  }
12
13  const handleDelete = async (event) => {
14    event.preventDefault();
15    setError("");
16
17    try {
18      await deleteRecord(recordId);
19      navigate("/");
20    } catch {
21      setError("Unable to delete record.");
22    }
23  };
24
25  return (
26    <div>
27      <h1>Delete Restaurant Inspection</h1>
28
29      {error && <p>{error}</p>}
30
31      <form onSubmit={handleDelete}>
32        <div>
33          <label htmlFor="recordId">Record ID</label>
34          <input
35            id="recordId"
36            type="number"
37            value={recordId}
38            onChange={(event) => setRecordId(event.target.value)}
39            required
40          />
41        </div>
42
43        <button type="submit">Delete Inspection</button>
```

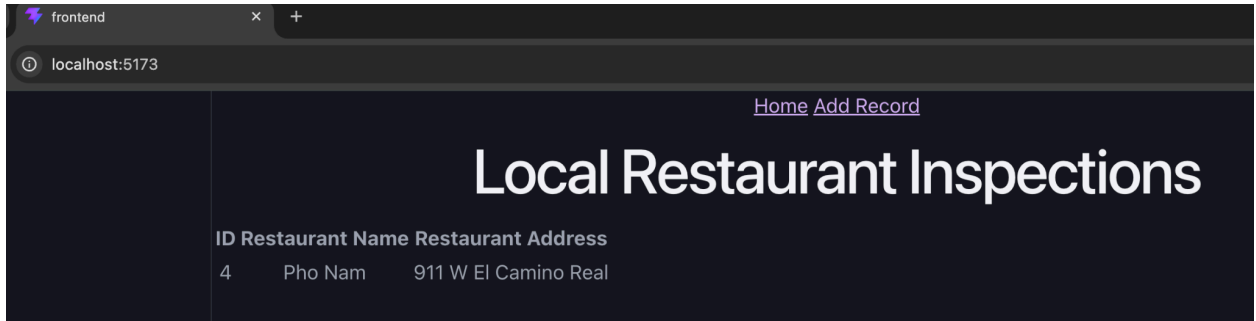
frontend x +

localhost:5173/delete

[Home](#) [Add Record](#)

Delete Restaurant Inspection

Record ID



V. For all the above components, make sure to:

The requirements in Part V are demonstrated in the code screenshots above. Props are passed to the Create, Update, and Delete components, React hooks are used where necessary, and React Router is used for routing.

Part 2 - MySQL Persistence and Server-Side Sessions

Create MySQL database: s9981_rel

```

optim — mysql -u root -p — 80x34

[mysql> CREATE DATABASE s9981_rel;
Query OK, 1 row affected (0.00 sec)

[mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| airbnb1  |
| AirlineDB |
| ecommercedb |
| flaskdemo |
| information_schema |
| mysql    |
| OpenFlights |
| p21_handshake |
| performance_schema |
| quizdb   |
| rental_platform |
| s9981_rel |
| sys      |
+-----+
13 rows in set (0.00 sec)

[mysql> USE s9981_rel;
Database changed
[mysql> SELECT DATABASE();
+-----+
| DATABASE() |
+-----+
| s9981_rel  |
+-----+
1 row in set (0.00 sec)

mysql>

```

Table: your primary domain entity (auto-increment id, primary field, secondary field)

```

optim — mysql -u root -p — 80x19

mysql> CREATE TABLE restaurant_inspections (
->   id INT AUTO_INCREMENT PRIMARY KEY,
->   restaurant_name VARCHAR(255) NOT NULL,
->   restaurant_address VARCHAR(255) NOT NULL
-> );
Query OK, 0 rows affected (0.01 sec)

[mysql> DESCRIBE restaurant_inspections;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id             | int           | NO   | PRI | NULL    | auto_increment |
| restaurant_name | varchar(255)  | NO   |     | NULL    |                |
| restaurant_address | varchar(255) | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql>

```

Also create tables for authentication/session management: users (id, name, email (unique), password_hash) and sessions (id (session token), user_id, created_at, expires_at)

```

optim — mysql -u root -p — 84x58

mysql> CREATE TABLE users (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   name VARCHAR(255) NOT NULL,
  ->   email VARCHAR(255) NOT NULL UNIQUE,
  ->   password_hash VARCHAR(255) NOT NULL
  -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE sessions (
  ->   id VARCHAR(255) PRIMARY KEY,
  ->   user_id INT NOT NULL,
  ->   created_at DATETIME NOT NULL,
  ->   expires_at DATETIME NOT NULL,
  ->   FOREIGN KEY (user_id) REFERENCES users(id)
  -> );
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_s9981_rel |
+-----+
| restaurant_inspections |
| sessions                |
| users                   |
+-----+
3 rows in set (0.00 sec)

mysql> DESCRIBE users;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL    | auto_increment |
| name       | varchar(255)  | NO   |     | NULL    |                |
| email      | varchar(255)  | NO   | UNI | NULL    |                |
| password_hash | varchar(255) | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> DESCRIBE sessions;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | varchar(255)  | NO   | PRI | NULL    |                |
| user_id    | int           | NO   | MUL | NULL    |                |
| created_at | datetime      | NO   |     | NULL    |                |
| expires_at | datetime      | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>

```

Add a new record (POST)

```
code > routers > records.py > delete_record
13
14     router = APIRouter(prefix="/records", tags=["records"])
15
16
17 @router.post("", response_model=RestaurantInspectionResponse)
18 def create_record(
19     record: RestaurantInspectionCreate,
20     db: DatabaseSession = Depends(get_db),
21     user: User = Depends(get_current_user),
22 ):
23     new_record = RestaurantInspection(
24         restaurant_name=record.restaurant_name,
25         restaurant_address=record.restaurant_address,
26     )
27
28     db.add(new_record)
29     db.commit()
30     db.refresh(new_record)
31
32     return new_record
33
```

POST Post data +

My Collection > Post data

POST http://127.0.0.1:8081/records Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {
2   "restaurant_name": "02 Valley",
3   "restaurant_address": "452 University Ave"
4 }
```

Body Cookies 1 Headers 4 Test Results 1/1 200 OK · 25 ms · 205 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 2,
3   "restaurant_name": "02 Valley",
4   "restaurant_address": "452 University Ave"
5 }
```

View all records (GET)

```
33
34
35 @router.get("", response_model=list[RestaurantInspectionResponse])
36 def get_records(
37     db: DatabaseSession = Depends(get_db),
38     user: User = Depends(get_current_user),
39 ):
40     return db.query(RestaurantInspection).all()
41
```

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8081/records`. The response is a 200 OK status with a response time of 391 ms and a body size of 207 B. The response body is displayed in JSON format, showing a list of restaurant inspection records.

GET `http://127.0.0.1:8081/records` **Send**

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

1

Body Cookies 1 Headers 4 Test Results 1/1 **200 OK** - 391 ms - 207 B

JSON Preview Visualize

```
1 [
2   {
3     "id": 2,
4     "restaurant_name": "02 Valley",
5     "restaurant_address": "452 University Ave"
6   }
7 ]
```

View a record by ID (GET)

```
41
42
43 @router.get("/{record_id}", response_model=RestaurantInspectionResponse)
44 def get_record(
45     record_id: int,
46     db: DatabaseSession = Depends(get_db),
47     user: User = Depends(get_current_user),
48 ):
49     record = (
50         db.query(RestaurantInspection)
51         .filter(RestaurantInspection.id == record_id)
52         .first()
53     )
54
55     if record is None:
56         raise HTTPException(
57             status_code=404,
58             detail="Record not found",
59         )
60
61     return record
62
```

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://127.0.0.1:8081/records/2
- Response Status:** 200 OK
- Response Time:** 26 ms
- Response Size:** 205 B
- Response Body (JSON):**

```
{
  "id": 2,
  "restaurant_name": "02 Valley",
  "restaurant_address": "452 University Ave"
}
```

Update record details (PUT)

```
62
63
64 @router.put("/{record_id}", response_model=RestaurantInspectionResponse)
65 def update_record(
66     record_id: int,
67     updated_record: RestaurantInspectionUpdate,
68     db: DatabaseSession = Depends(get_db),
69     user: User = Depends(get_current_user),
70 ):
71     record = (
72         db.query(RestaurantInspection)
73         .filter(RestaurantInspection.id == record_id)
74         .first()
75     )
76
77     if record is None:
78         raise HTTPException(
79             status_code=404,
80             detail="Record not found",
81         )
82
83     record.restaurant_name = updated_record.restaurant_name
84     record.restaurant_address = updated_record.restaurant_address
85
86     db.commit()
87     db.refresh(record)
88
89     return record
90
91
```

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** http://127.0.0.1:8081/records/2
- Body (raw):**

```
{
  "restaurant_name": "02 Valley",
  "restaurant_address": "452 University Avenue"
}
```
- Status:** 200 OK
- Response Time:** 172 ms
- Response Size:** 208 B
- Response Body (JSON):**

```
{
  "id": 2,
  "restaurant_name": "02 Valley",
  "restaurant_address": "452 University Avenue"
}
```

Delete a record (DELETE)

```
91
92 @router.delete("/{record_id}")
93 def delete_record(
94     record_id: int,
95     db: DatabaseSession = Depends(get_db),
96     user: User = Depends(get_current_user),
97 ):
98     record = (
99         db.query(RestaurantInspection)
100         .filter(RestaurantInspection.id == record_id)
101         .first()
102     )
103
104     if record is None:
105         raise HTTPException(
106             status_code=404,
107             detail="Record not found",
108         )
109
110     db.delete(record)
111     db.commit()
112
113     return {"message": "Record deleted"}
```

The screenshot shows a REST client interface with a tab titled "DEL Post data". The URL bar shows "http://127.0.0.1:8081/records/2" and the method is set to "DELETE". The "Body" tab is selected, showing a JSON response:

```
{  "message": "Record deleted"}
```

. The status bar at the bottom indicates a "200 OK" response with a response time of 25 ms and a body size of 153 B. The interface also includes tabs for "Docs", "Params", "Authorization", "Headers", "Body", "Scripts", and "Settings".

Database

optim — mysql -u root -p — 86x52

mysql> USE s9981_rel;

Database changed

mysql>

mysql> SHOW TABLES;

```
+-----+
| Tables_in_s9981_rel |
+-----+
| restaurant_inspections |
| sessions               |
| users                  |
+-----+
```

3 rows in set (0.00 sec)

mysql>

mysql> DESCRIBE restaurant_inspections;

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
restaurant_name	varchar(255)	NO		NULL	
restaurant_address	varchar(255)	NO		NULL	

3 rows in set (0.00 sec)

mysql> DESCRIBE users;

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(255)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
password_hash	varchar(255)	NO		NULL	

4 rows in set (0.00 sec)

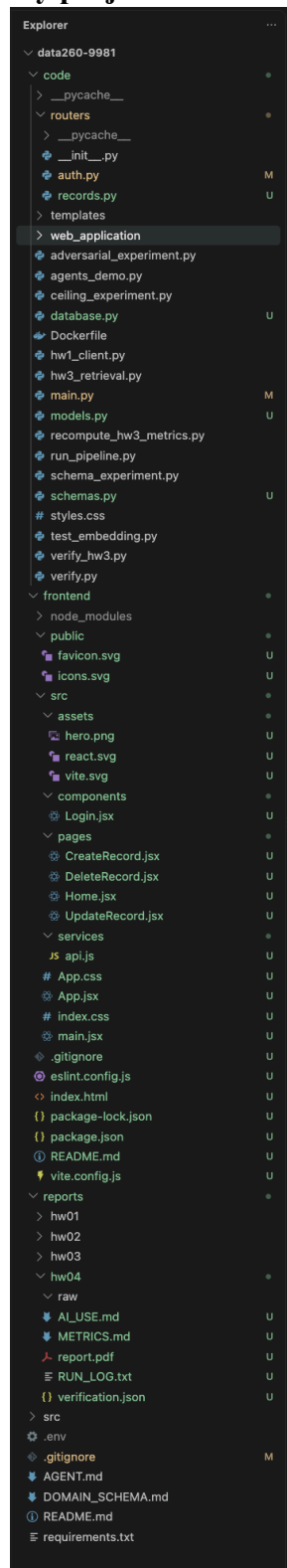
mysql> DESCRIBE sessions;

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI	NULL	
user_id	int	NO	MUL	NULL	
created_at	datetime	NO		NULL	
expires_at	datetime	NO		NULL	

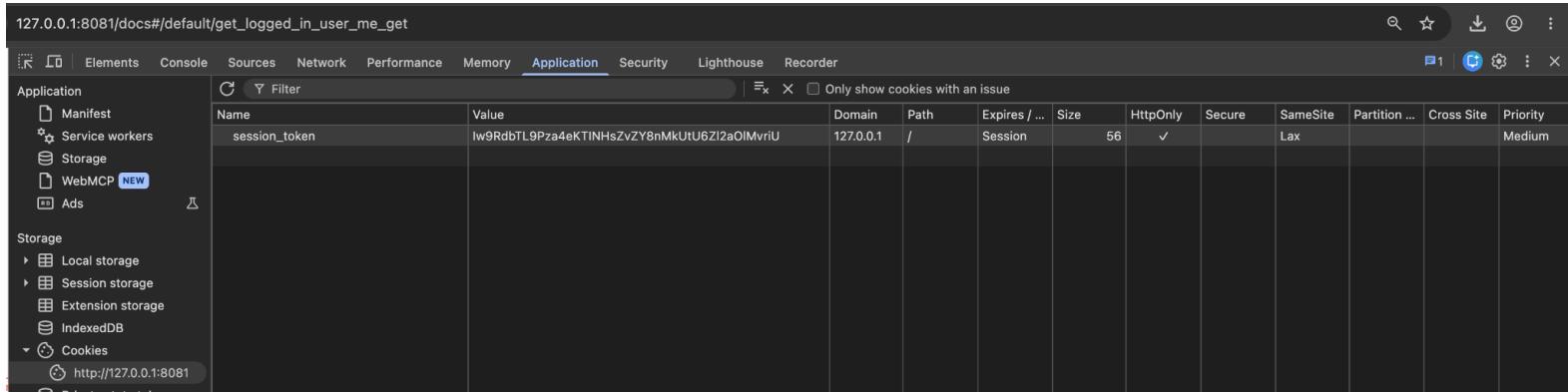
4 rows in set (0.00 sec)

mysql> █

My project folder structure



The session token issued at login must be stored server-side in the sessions table and referenced only by an opaque token in the browser's HTTP-only cookie - the cookie itself should never store user data directly.



127.0.0.1:8081/docs#/default/get_logged_in_user_me_get

Application

Filter

Only show cookies with an issue

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition ...	Cross Site	Priority
session_token	Iw9RdbTL9Pza4eKTINHsZvZY8nMkUtU6Zl2aOlMvriU	127.0.0.1	/	Session	56	✓		Lax			Medium

```
optim — mysql -u root -p — 80x24

mysql> SELECT id, user_id, created_at, expires_at
-> FROM sessions;

+-----+-----+-----+-----+
| id                | user_id | created_at          | expires_at          |
+-----+-----+-----+-----+
| Iw9RdbTL9Pza4eKTINHsZvZY8nMkUtU6Zl2aOlMvriU | 1       | 2026-09-28 07:58:37 | 2026-09-28 08:58:37 |
+-----+-----+-----+-----+

1 row in set (0.00 sec)

mysql>
```

Part 3 - N+1 Measurement and Query Tuning

1 Seed 5,000 rows of your primary domain entity and 200 related rows, using your SEED value. This related table is just test data for now - you'll build a proper related entity with full CRUD in a later homework. Commit your generator script and the schema/migration you used to create these rows.

part3_nplus1.sql

code > migrations > part3_nplus1.sql

```
1 CREATE TABLE IF NOT EXISTS inspection_notes (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     restaurant_inspection_id INT NOT NULL,  
4     note VARCHAR(255) NOT NULL,  
5     FOREIGN KEY (restaurant_inspection_id)  
6         REFERENCES restaurant_inspections(id)  
7         ON DELETE CASCADE  
8 );
```

seed_part3.py

code > seed_part3.py > main

```
6 SEED = 9981  
7 PRIMARY_ROW_COUNT = 5000  
8 RELATED_ROW_COUNT = 200
```

Problems Output Debug Console Terminal Ports

```
• (base) optim@Timothys-MacBook-Pro data260-9981 % cd /Users/optim/Desktop/260/data260-9981/code  
python seed_part3.py  
SEED = 9981  
restaurant_inspections rows = 5000  
inspection_notes rows = 200  
○ (base) optim@Timothys-MacBook-Pro code %
```

```
optim — mysql -u root -p — 86x52

mysql> SELECT COUNT(*) FROM restaurant_inspections;
+-----+
| COUNT(*) |
+-----+
|      5000 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT COUNT(*) FROM inspection_notes;
+-----+
| COUNT(*) |
+-----+
|       200 |
+-----+
1 row in set (0.00 sec)

[mysql> SELECT * FROM inspection_notes LIMIT 10;
+-----+-----+-----+
| id | restaurant_inspection_id | note |
+-----+-----+-----+
| 1 | 7192 | Inspection note 1 |
| 2 | 5794 | Inspection note 2 |
| 3 | 5128 | Inspection note 3 |
| 4 | 6321 | Inspection note 4 |
| 5 | 8674 | Inspection note 5 |
| 6 | 8766 | Inspection note 6 |
| 7 | 8206 | Inspection note 7 |
| 8 | 7396 | Inspection note 8 |
| 9 | 7767 | Inspection note 9 |
| 10 | 7318 | Inspection note 10 |
+-----+-----+-----+
10 rows in set (0.00 sec)

mysql> █
```

2 The 200 related rows must be associated with the primary domain entity, and the list endpoint's response must include this related data.

```
code > routers > records.py > get_record

43 @router.get(
44     "/part3/nplus1",
45     response_model=list[RestaurantInspectionWithNotesResponse],
46 )
47 def get_records_with_notes_nplus1(
48     page_size: int = 10,
49     db: DatabaseSession = Depends(get_db),
50     user: User = Depends(get_current_user),
51 ):
52     records = (
53         db.query(RestaurantInspection)
54         .order_by(RestaurantInspection.id)
55         .limit(page_size)
56         .all()
57     )
58
59     result = []
60
61     for record in records:
62         notes = (
63             db.query(InspectionNote)
64             .filter(
65                 InspectionNote.restaurant_inspection_id == record.id
66             )
67             .all()
68         )
69
70         result.append(
71             {
72                 "id": record.id,
73                 "restaurant_name": record.restaurant_name,
74                 "restaurant_address": record.restaurant_address,
75                 "notes": notes,
76             }
77         )
78
79     return result
80
```

GET Post data

No Environment

My Collection > Post data

SaveShare

GEThttp://127.0.0.1:8081/records/part3/nplus1?page_size=10Send

DocsParamsAuthorizationHeaders10BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
✓ page_size	10		
Key	Value	Description	

BodyCookies1Headers4Test Results1/1

200 OK40 ms1.13 KBSave Response

JSONPreviewVisualize

```
1  [
2    {
3      "id": 10005,
4      "restaurant_name": "Restaurant 1",
5      "restaurant_address": "101 Test Street",
6      "notes": []
7    },
8    {
9      "id": 10006,
10     "restaurant_name": "Restaurant 2",
11     "restaurant_address": "102 Test Street",
12     "notes": []
13   },
14   {
15     "id": 10007,
16     "restaurant_name": "Restaurant 3",
17     "restaurant_address": "103 Test Street",
18     "notes": []
19   },
20   {
21     "id": 10008,
22     "restaurant_name": "Restaurant 4",
23     "restaurant_address": "104 Test Street",
24     "notes": [
25       {
26         "id": 268,
27         "restaurant_inspection_id": 10008,
28         "note": "Inspection note 68"
29       }
30     ]
31   }
32 ]
```

Fix the status code assertionDesktop AgentGlobalsVaultTools

3 Build a list endpoint that runs one extra query per record to fetch its related data (this is the classic "N+1" problem - intentionally naive for now).

```
records.py U X
code > routers > records.py > get_record
47 def get_records_with_notes_nplus1(
52     records = (
53         db.query(RestaurantInspection)
54         .order_by(RestaurantInspection.id)
55         .limit(page_size)
56         .all()
57     )
58
59     result = []
60
61     for record in records:
62         notes = (
63             db.query(InspectionNote)
64             .filter(
65                 InspectionNote.restaurant_inspection_id == record.id
66             )
67             .all()
68         )
69
70         result.append(
71             {
72                 "id": record.id,
73                 "restaurant_name": record.restaurant_name,
74                 "restaurant_address": record.restaurant_address,
75                 "notes": notes,
76             }
77         )
78
79     return result
```

The endpoint first executes one query to retrieve the requested restaurant inspections. It then loops through each record and executes one additional query to retrieve that record's related inspection notes. Therefore, a page containing N records triggers approximately N+1 SQL queries, intentionally demonstrating the N+1 query problem.

4 Test this endpoint at page sizes 10, 50, and 200. For each size, run 30 requests and record how many SQL queries each request triggers, plus the p50/p95/p99 latency.

```

records.py U X
code > routers > records.py > get_records_with_notes_fixed
78 def get_records_with_notes_nplus1(
83     records = (
84         db.query(RestaurantInspection)
85         .order_by(RestaurantInspection.id)
86         .limit(page_size)
87         .all()
88     )
89
90     result = []
91
92     for record in records:
93         notes = (
94             db.query(InspectionNote)
95             .filter(
96                 InspectionNote.restaurant_inspection_id == record.id
97             )
98             .all()
99         )

```

```

Problems Output Debug Console Terminal Ports
• (base) optim@Timothys-MacBook-Pro data260-9981 % cd /Users/optim/Desktop/260/data260-9981/code
python benchmark_nplus1.py
Page size: 10
Queries per request: 13-13
p50 latency: 3.74 ms
p95 latency: 4.95 ms
p99 latency: 7.12 ms

Page size: 50
Queries per request: 53-53
p50 latency: 8.21 ms
p95 latency: 10.85 ms
p99 latency: 10.88 ms

Page size: 200
Queries per request: 203-203
p50 latency: 27.75 ms
p95 latency: 29.28 ms
p99 latency: 29.69 ms

Saved 90 raw requests to:
/Users/optim/Desktop/260/data260-9981/reports/hw04/raw/nplus1_naive.csv
○ (base) optim@Timothys-MacBook-Pro code %

```

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	13	3.74	4.95	7.12
50	naive	53	8.21	10.85	10.88
200	naive	203	27.75	29.28	29.69

Each page size was tested with 30 requests. The naive endpoint triggered 13, 53, and 203 SQL statements per request for page sizes 10, 50, and 200, respectively. This includes two authentication queries, one query to fetch the primary restaurant inspection records, and one additional query per returned record to fetch its related inspection notes. As the page size increased, both the SQL query count and request latency increased because the endpoint performs one related-data query for every primary record.

5 Now fix the N+1 issue using a join or eager loading, and re-run the same measurements.

```
records.py U X
code > routers > records.py > get_records_with_notes_fixed
52 def get_records_with_notes_fixed(
57     records = (
58         db.query(RestaurantInspection)
59         .options(joinedload(RestaurantInspection.notes))
60         .order_by(RestaurantInspection.id)
61         .limit(page_size)
62         .all()
63     )
```

```
Problems Output Debug Console Terminal Ports
• (base) optim@Timothys-MacBook-Pro code % cd /Users/optim/Desktop/260/data260-9981/code
python benchmark_fixed.py
Page size: 10
Queries per request: 3-3
p50 latency: 2.59 ms
p95 latency: 3.39 ms
p99 latency: 4.89 ms

Page size: 50
Queries per request: 3-3
p50 latency: 2.45 ms
p95 latency: 2.81 ms
p99 latency: 2.85 ms

Page size: 200
Queries per request: 3-3
p50 latency: 3.89 ms
p95 latency: 4.72 ms
p99 latency: 6.33 ms

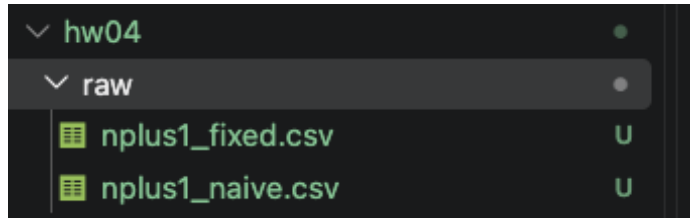
Saved 90 raw requests to:
/Users/optim/Desktop/260/data260-9981/reports/hw04/raw/nplus1_fixed.csv
○ (base) optim@Timothys-MacBook-Pro code %
```

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	13	3.74	4.95	7.12
10	fixed	3	2.59	3.39	4.89
50	naive	53	8.21	10.85	10.88
50	fixed	3	2.45	2.81	2.85
200	naive	203	27.75	29.28	29.69
200	fixed	3	3.89	4.72	6.33

The N+1 issue was fixed using SQLAlchemy eager loading with `joinedload()`. The fixed endpoint retrieves the primary restaurant inspection records and their related inspection notes in one joined query rather than issuing one additional related-data query per record. Including the two authentication queries, the fixed endpoint consistently triggered 3 SQL statements per request at page sizes 10, 50, and 200.

6 Save the raw data for all 180 requests ($3 \text{ page sizes} \times 2 \text{ versions} \times 30 \text{ requests}$) in `reports/hw04/raw/`, so your results can be double-checked.

The raw measurements for all 180 requests were saved under `reports/hw04/raw/`. The naive endpoint results are stored in `nplus1_naive.csv`, and the fixed endpoint results are stored in `nplus1_fixed.csv`. Each file contains 90 requests: 30 requests each for page sizes 10, 50, and 200.



```

nplust_naive.csv U X
reports > hw04 > raw > nplust_naive.csv
1 version,page_size,run,query_count,latency_ms
2 naive,10,1,13,7.99412501502327
3 naive,10,2,13,4.906417016172782
4 naive,10,3,13,4.9873749958351254
5 naive,10,4,13,4.353291995357722
6 naive,10,5,13,3.879750001942739
7 naive,10,6,13,3.6729580024257302
8 naive,10,7,13,3.6218340101186186
9 naive,10,8,13,3.5708340001292527
10 naive,10,9,13,4.002000001491979
11 naive,10,10,13,4.3677920184563845
12 naive,10,11,13,4.17112500872463
13 naive,10,12,13,4.4622500136028975
14 naive,10,13,13,4.080749989952892
15 naive,10,14,13,3.872000001138076
16 naive,10,15,13,3.9797499775886536
17 naive,10,16,13,4.126165993511677
18 naive,10,17,13,4.019417014205828
19 naive,10,18,13,3.81504200049676
20 naive,10,19,13,3.5565419821068645
21 naive,10,20,13,3.5324999771546572
22 naive,10,21,13,3.5163339925929904
23 naive,10,22,13,3.423167014261077
24 naive,10,23,13,3.657499995473772
25 naive,10,24,13,3.453582990914583
26 naive,10,25,13,3.4612919844221324
27 naive,10,26,13,3.426791983656585
28 naive,10,27,13,3.5762089828494936
29 naive,10,28,13,3.468999988399446
30 naive,10,29,13,3.437041013967246
31 naive,10,30,13,3.3418839969292283
32 naive,50,1,53,10.88024998898618
33 naive,50,2,53,10.835833003511652
34 naive,50,3,53,10.628209012793377
35 naive,50,4,53,10.36095802555792
36 naive,50,5,53,10.430124995764345
37 naive,50,6,53,10.261832998367026
38 naive,50,7,53,10.862166993319988
39 naive,50,8,53,10.673041979316622
40 naive,50,9,53,10.470625013113022
41 naive,50,10,53,10.08545799413696
42 naive,50,11,53,8.170749992132187
43 naive,50,12,53,8.242250012699515
44 naive,50,13,53,8.075916994130239
45 naive,50,14,53,8.147832995746285
46 naive,50,15,53,8.037499996135011
47 naive,50,16,53,8.09849999495782
48 naive,50,17,53,8.578165987273678
49 naive,50,18,53,8.18450000951998
50 naive,50,19,53,8.141883002556115
51 naive,50,20,53,7.886041013989598
52 naive,50,21,53,7.936082984087989
53 naive,50,22,53,7.993708975845948
54 naive,50,23,53,8.46220800303854
55 naive,50,24,53,8.246082987170666
56 naive,50,25,53,8.06520800688304
57 naive,50,26,53,7.92033298057504
58 naive,50,27,53,7.965875003719702
59 naive,50,28,53,7.8993750212248415
60 naive,50,29,53,7.985083997482434
61 naive,50,30,53,8.2834999775514
62 naive,200,1,203,28.17554198554717
63 naive,200,2,203,28.166874981252477
64 naive,200,3,203,27.51433299272321
65 naive,200,4,203,27.626832976238802
66 naive,200,5,203,27.364791982108727
67 naive,200,6,203,27.006458985852078
68 naive,200,7,203,26.89700000337325
69 naive,200,8,203,27.167958003701642
70 naive,200,9,203,27.78304199455306
71 naive,200,10,203,27.26395899662748
72 naive,200,11,203,27.524791978066787
73 naive,200,12,203,26.919708005152643
74 naive,200,13,203,27.299500012304634
75 naive,200,14,203,28.15962498425506
76 naive,200,15,203,28.065583028364927
77 naive,200,16,203,28.04858301533386
78 naive,200,17,203,28.00054199178703
79 naive,200,18,203,27.4785409856122
80 naive,200,19,203,27.89820800535381
81 naive,200,20,203,27.49770801165141
82 naive,200,21,203,28.41479200287722
83 naive,200,22,203,28.957874979823828
84 naive,200,23,203,29.544166987761855
85 naive,200,24,203,29.746499989414588
86 naive,200,25,203,27.83441697829403
87 naive,200,26,203,28.581708000274375
88 naive,200,27,203,27.861125010531396
89 naive,200,28,203,27.23662499920465
90 naive,200,29,203,27.70837501157075
91 naive,200,30,203,27.456000010715798
92

```

```

nplust_fixed.csv U X
reports > hw04 > raw > nplust_fixed.csv
1 version,page_size,run,query_count,latency_ms
2 fixed,10,1,3,5.464292014949024
3 fixed,10,2,3,3.471083997283131
4 fixed,10,3,3,2.9322499758563936
5 fixed,10,4,3,3.0095000111032277
6 fixed,10,5,3,3.1866249919403344
7 fixed,10,6,3,3.1414580007549375
8 fixed,10,7,3,2.9748749802820385
9 fixed,10,8,3,3.28341600834392
10 fixed,10,9,3,2.9252920066937804
11 fixed,10,10,3,2.823374990839511
12 fixed,10,11,3,2.562624984420836
13 fixed,10,12,3,2.465291996486485
14 fixed,10,13,3,3.0250409909058362
15 fixed,10,14,3,2.4324999831151217
16 fixed,10,15,3,2.335458993911743
17 fixed,10,16,3,2.2922499920241535
18 fixed,10,17,3,2.547041978687048
19 fixed,10,18,3,2.493874984793365
20 fixed,10,19,3,2.33499999740161
21 fixed,10,20,3,2.5582080124877393
22 fixed,10,21,3,2.2721249843016267
23 fixed,10,22,3,2.315749996341765
24 fixed,10,23,3,2.37574998755008
25 fixed,10,24,3,2.593457989860326
26 fixed,10,25,3,2.642084000399336
27 fixed,10,26,3,2.5847500073723495
28 fixed,10,27,3,2.5325829919893295
29 fixed,10,28,3,2.4807500012684613
30 fixed,10,29,3,2.7119579899590462
31 fixed,10,30,3,2.6174169906880707
32 fixed,50,1,3,2.3657499987166375
33 fixed,50,2,3,2.3358330072369426
34 fixed,50,3,3,2.445000020088628
35 fixed,50,4,3,2.7612920093815774
36 fixed,50,5,3,2.8492500132415444
37 fixed,50,6,3,2.617457997985184
38 fixed,50,7,3,2.4168340023607016
39 fixed,50,8,3,2.8494169819168746
40 fixed,50,9,3,2.740833006100729
41 fixed,50,10,3,2.5202079850714654
42 fixed,50,11,3,2.4530419905204326
43 fixed,50,12,3,2.441124990582466
44 fixed,50,13,3,2.423917001578957
45 fixed,50,14,3,2.458958013448864
46 fixed,50,15,3,2.7603340276982635
47 fixed,50,16,3,2.4702500086277723
48 fixed,50,17,3,2.345000015338883
49 fixed,50,18,3,2.5284159928560257
50 fixed,50,19,3,2.4360829847864807
51 fixed,50,20,3,2.512250008294359
52 fixed,50,21,3,2.5597500207368284
53 fixed,50,22,3,2.564583992352709
54 fixed,50,23,3,2.437457995256409
55 fixed,50,24,3,2.551374986069277
56 fixed,50,25,3,2.4140829918906093
57 fixed,50,26,3,2.339458995265886
58 fixed,50,27,3,2.4234579759649932
59 fixed,50,28,3,2.3451670131118043
60 fixed,50,29,3,2.2250420006457716
61 fixed,50,30,3,2.358416997594759
62 fixed,200,1,3,3.6181670147925615
63 fixed,200,2,3,3.428625001106411
64 fixed,200,3,3,4.050874995300546
65 fixed,200,4,3,3.971999976783991
66 fixed,200,5,3,3.9075409877114
67 fixed,200,6,3,3.7552920111920685
68 fixed,200,7,3,4.266458010533825
69 fixed,200,8,3,3.6225829971954226
70 fixed,200,9,3,3.769167000427842
71 fixed,200,10,3,3.8162080163601786
72 fixed,200,11,3,3.7240000002424452
73 fixed,200,12,3,3.705374983837828
74 fixed,200,13,3,3.6589590017683804
75 fixed,200,14,3,3.540375008014962
76 fixed,200,15,3,3.873499983455986
77 fixed,200,16,3,4.735166992759332
78 fixed,200,17,3,6.975209020311013
79 fixed,200,18,3,4.460875003132969
80 fixed,200,19,3,4.6952919801697135
81 fixed,200,20,3,4.163249977864325
82 fixed,200,21,3,3.868166997563094
83 fixed,200,22,3,4.268374992534518
84 fixed,200,23,3,4.326333000790328
85 fixed,200,24,3,4.033459001220763
86 fixed,200,25,3,4.111665999516845
87 fixed,200,26,3,3.764416993362829
88 fixed,200,27,3,4.063625005073845
89 fixed,200,28,3,3.8007920084055513
90 fixed,200,29,3,4.138959018746391
91 fixed,200,30,3,3.778708021854982
92

```

7 Report how much faster the fixed version is at each page size, and explain why the speed-up changes as the page size grows.

Page size	Naive p50	Fixed p50	Speed-up	Latency reduction
10	3.74 ms	2.59 ms	1.44×	30.75%
50	8.21 ms	2.45 ms	3.35×	70.16%
200	27.75 ms	3.89 ms	7.13×	85.98%

The fixed version became increasingly faster as the page size grew. At page size 10, the fixed endpoint was approximately 1.44× faster based on p50 latency, reducing latency from 3.74 ms to 2.59 ms. At page size 50, it was approximately 3.35× faster, reducing p50 latency from 8.21 ms to 2.45 ms. At page size 200, it was approximately 7.13× faster, reducing p50 latency from 27.75 ms to 3.89 ms. The speed-up increases with page size because the naive implementation performs one additional related-data query for every returned record, causing its SQL query count to grow from 13 to 53 to 203. In contrast, the fixed endpoint uses eager loading and consistently performs only 3 SQL statements per request, including authentication. As the number of returned records increases, the cost of the naive N+1 queries grows substantially while the fixed version avoids that additional database round-trip overhead.

8 Add one index to your database, and show the EXPLAIN output before and after adding it - explain what changed.

```

mysql> USE #9981_re1;
Database changed
mysql> EXPLAIN
mysql> -> SELECT *
-> FROM restaurant_inspections
-> WHERE restaurant_address = '101 Test Street';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | restaurant_inspections | NULL | ALL | NULL | NULL | NULL | NULL | 5000 | 10.00 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> CREATE INDEX idx_restaurant_inspections_address
-> ON restaurant_inspections (restaurant_address);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SHOW INDEX FROM restaurant_inspections;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non-unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| restaurant_inspections | 0 | PRIMARY | 1 | id | A | 5000 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| restaurant_inspections | 1 | idx_restaurant_inspections_address | 1 | restaurant_address | A | 5000 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> EXPLAIN
mysql> -> SELECT *
-> FROM restaurant_inspections
-> WHERE restaurant_address = '101 Test Street';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | restaurant_inspections | NULL | ref | idx_restaurant_inspections_address | idx_restaurant_inspections_address | 1022 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>

```

Before adding the index, MySQL performed a full table scan on restaurant_inspections, shown by type = ALL, with no possible or selected index and an estimated 5,000 rows examined. After creating idx_restaurant_inspections_address on restaurant_address, the same query used the new index, shown by type = ref and key = idx_restaurant_inspections_address. The estimated number of rows examined dropped from 5,000 to 1, demonstrating that MySQL could directly locate the matching restaurant address instead of scanning the entire table.

9 Include Postman screenshots showing both versions of the endpoint working correctly at each page size.

Page Size 10 Naive

The screenshot shows a Postman interface for a GET request. The URL is `http://127.0.0.1:8081/records/part3/nplus1?page_size=10`. The request is successful, returning a `200 OK` status with a response time of 21 ms and a body size of 1.16 KB. The response body is in JSON format, showing two restaurant records. The first record has an ID of 10008, a name of "Restaurant 4", and an address of "104 Test Street". It also has a list of notes, including one with ID 268, restaurant inspection ID 10008, and the text "Inspection note 68". The second record has an ID of 10009, a name of "Restaurant 5", and an address of "105 Test Street".

Key	Value	Description	Bulk Edit
✓ page_size	10		
Key	Value	Description	

```
20 {
21   "id": 10008,
22   "restaurant_name": "Restaurant 4",
23   "restaurant_address": "104 Test Street",
24   "notes": [
25     {
26       "id": 268,
27       "restaurant_inspection_id": 10008,
28       "note": "Inspection note 68"
29     }
30   ],
31 },
32 {
33   "id": 10009,
34   "restaurant_name": "Restaurant 5",
35   "restaurant_address": "105 Test Street",
36   "notes": []
37 },
```

Page Size 10 Fixed

GET Post data

My Collection > Post data

GET http://127.0.0.1:8081/records/part3/fixed?page_size=10

Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
✓ page_size	10		
Key	Value	Description	

Body Cookies 1 Headers 5 Test Results 1/1

200 OK - 18 ms - 1.15 KB

Save Response

{ } JSON Preview Visualize

```
20 {
21   "id": 10008,
22   "restaurant_name": "Restaurant 4",
23   "restaurant_address": "104 Test Street",
24   "notes": [
25     {
26       "id": 268,
27       "restaurant_inspection_id": 10008,
28       "note": "Inspection note 68"
29     }
30   ],
31 },
32 {
33   "id": 10009,
34   "restaurant_name": "Restaurant 5",
35   "restaurant_address": "105 Test Street",
36   "notes": []
37 },
38 {
```

Page Size 50 Naive

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:8081/records/part3/nplus?page_size=50`. The response is a 200 OK status with a 44 ms response time and 5.01 KB of data. The response body is displayed in JSON format, showing a list of restaurant records. The first record is for Restaurant 4, and the second is for Restaurant 5. The response is paginated, showing only the first two records out of a total of 100.

Query Params

Key	Value	Description	Bulk Edit
✓ page_size	50		
Key	Value	Description	

Body

```
20 {
21   "id": 10008,
22   "restaurant_name": "Restaurant 4",
23   "restaurant_address": "104 Test Street",
24   "notes": [
25     {
26       "id": 268,
27       "restaurant_inspection_id": 10008,
28       "note": "Inspection note 68"
29     }
30   ],
31 },
32 {
33   "id": 10009,
34   "restaurant_name": "Restaurant 5",
35   "restaurant_address": "105 Test Street",
36   "notes": []
37 },
38 }
```

Page Size 50 Fixed

GET Post data

My Collection > Post data

GET Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ page_size	50			
Key	Value	Description		

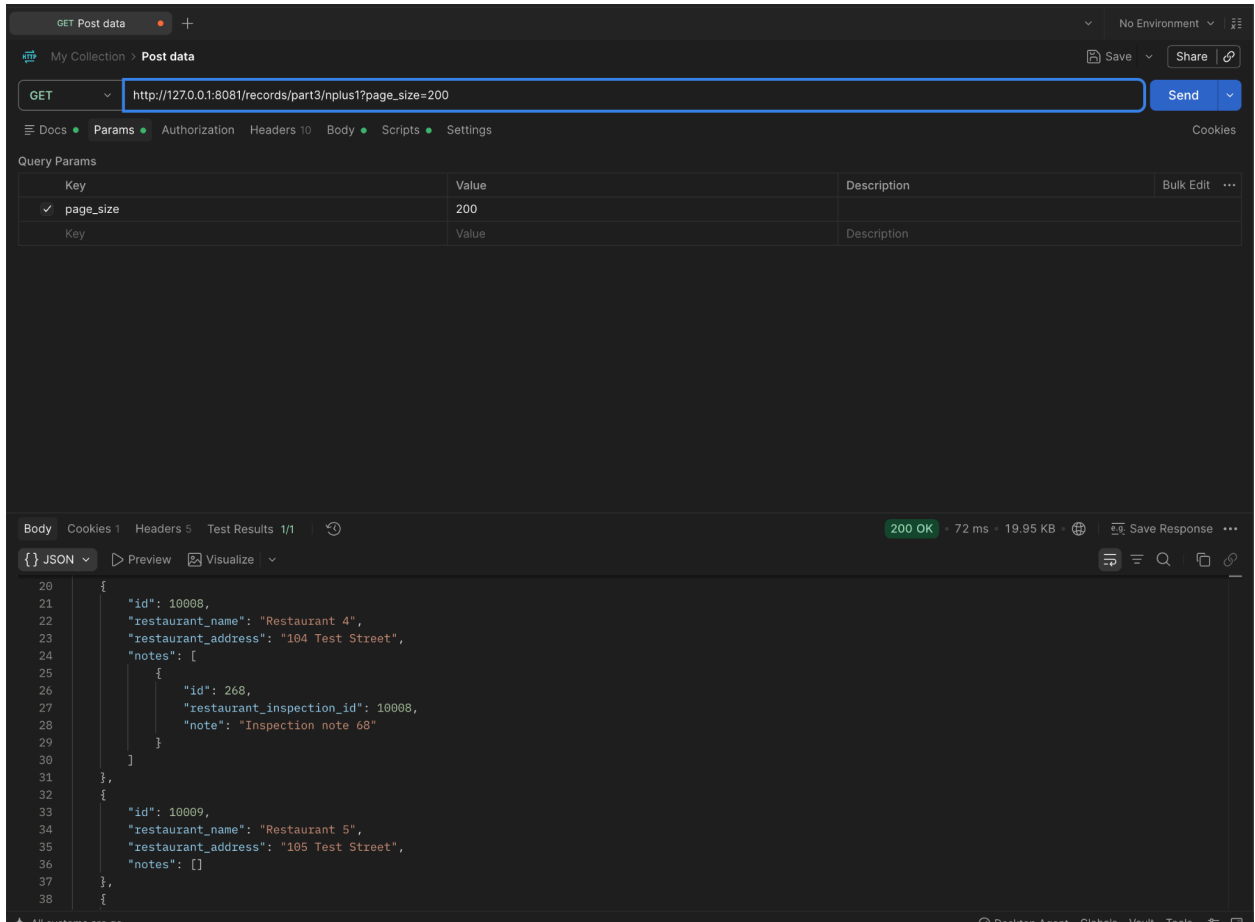
Body Cookies 1 Headers 5 Test Results 1/1

200 OK • 16 ms • 5.01 KB

JSON Preview Visualize

```
20 {
21   "id": 10008,
22   "restaurant_name": "Restaurant 4",
23   "restaurant_address": "104 Test Street",
24   "notes": [
25     {
26       "id": 268,
27       "restaurant_inspection_id": 10008,
28       "note": "Inspection note 68"
29     }
30   ],
31 },
32 {
33   "id": 10009,
34   "restaurant_name": "Restaurant 5",
35   "restaurant_address": "105 Test Street",
36   "notes": []
37 },
```

Page Size 200 Naive



GET Post data

My Collection > Post data

GET Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ page_size	200			
Key	Value	Description		

Body Cookies 1 Headers 5 Test Results 1/1

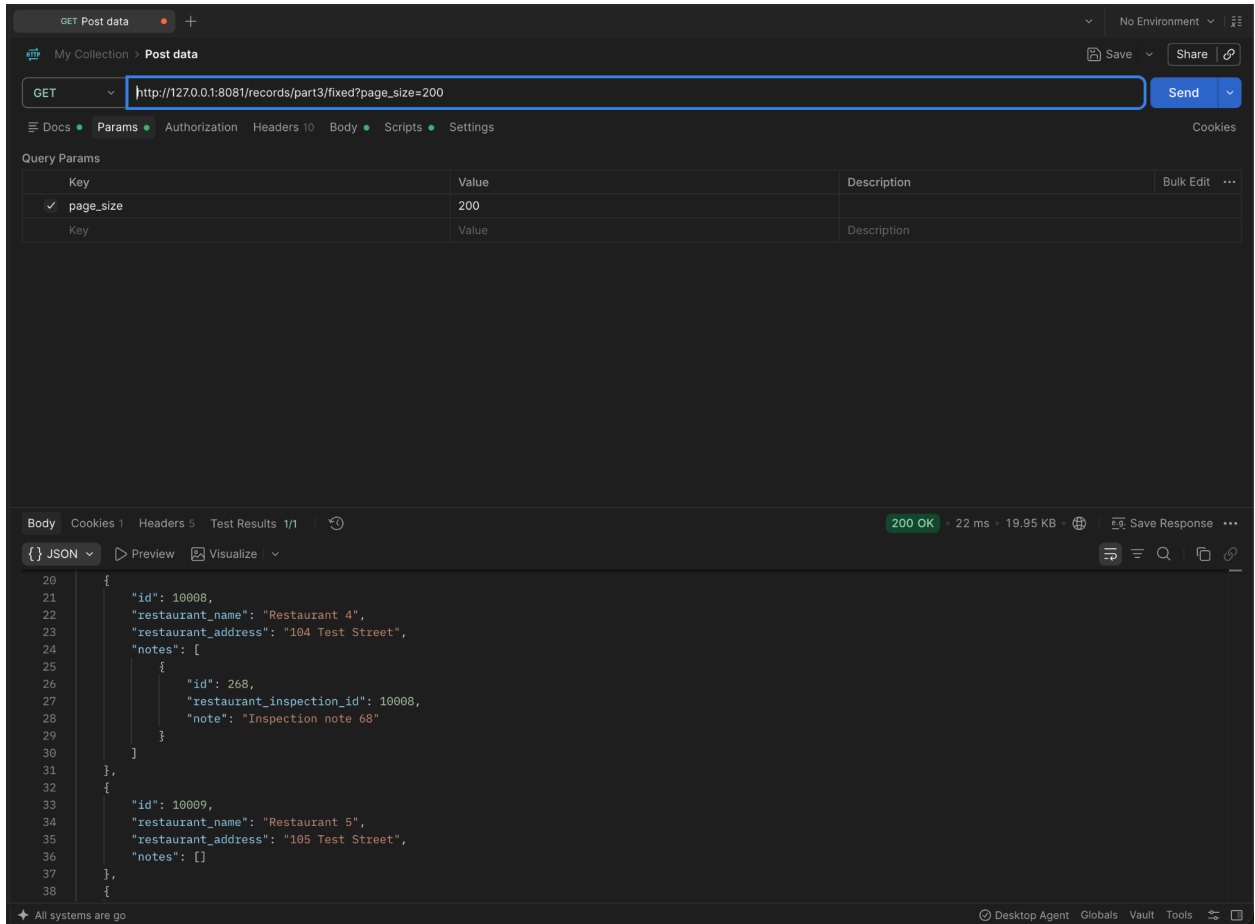
200 OK · 72 ms · 19.95 KB

Save Response

JSON Preview Visualize

```
29 {
21   "id": 10008,
22   "restaurant_name": "Restaurant 4",
23   "restaurant_address": "104 Test Street",
24   "notes": [
25     {
26       "id": 268,
27       "restaurant_inspection_id": 10008,
28       "note": "Inspection note 68"
29     }
30   ]
31 },
32 {
33   "id": 10009,
34   "restaurant_name": "Restaurant 5",
35   "restaurant_address": "105 Test Street",
36   "notes": []
37 },
38 {
```

Page Size 200 Fixed



The screenshot shows the Postman interface for a GET request. The URL is `http://127.0.0.1:8081/records/part3/fixed?page_size=200`. The query parameters table shows `page_size` with a value of `200`. The response status is `200 OK` with a response time of `22 ms` and a size of `19.95 KB`. The response body is a JSON array of two restaurant inspection records.

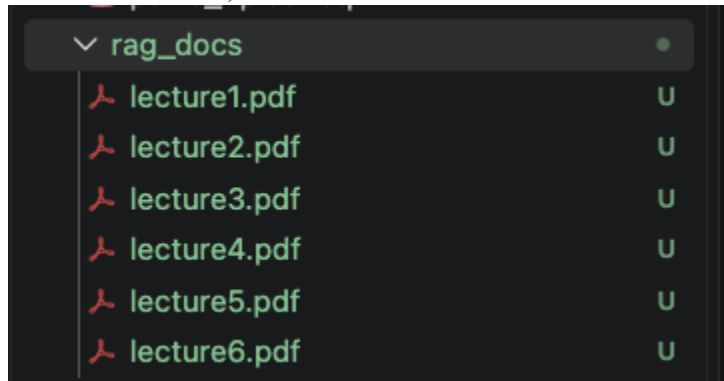
Key	Value	Description	Bulk Edit
✓ <code>page_size</code>	<code>200</code>		
Key	Value	Description	

```
{
  "id": 10008,
  "restaurant_name": "Restaurant 4",
  "restaurant_address": "104 Test Street",
  "notes": [
    {
      "id": 268,
      "restaurant_inspection_id": 10008,
      "note": "Inspection note 68"
    }
  ]
},
{
  "id": 10009,
  "restaurant_name": "Restaurant 5",
  "restaurant_address": "105 Test Street",
  "notes": []
},
{
  "id": 10010,
  "restaurant_name": "Restaurant 6",
  "restaurant_address": "106 Test Street",
  "notes": []
}
```

Postman confirmed that both the naive and fixed endpoints returned the expected restaurant inspection records and related inspection-note data at page sizes 10, 50, and 200.

Part 4 - Build a Grounded RAG Question-Answering System

1. Corpus and index: use at least 5 documents (course notes, policies, or technical docs). Chunk them (start with `chunk_size = 500`, `chunk_overlap = 50`), storing each chunk's text, source name, and `chunk_id`; embed the chunks and load them into a vector store (FAISS, Chroma, or LlamaIndex `VectorStoreIndex`).



 rag.py 1, U 

```
code > rag.py > chunk_text
8  CHUNK_SIZE = 500
9  CHUNK_OVERLAP = 50
10
11  DOCS_DIR = Path(__file__).resolve().parent / "rag_docs"
12  CHROMA_DIR = Path(__file__).resolve().parent / "chroma_db"
13
14  EMBED_MODEL = "nomic-embed-text"
15  OLLAMA_URL = "http://127.0.0.1:11434/api/embed"
16
```

 rag.py 1, U 

```
code > rag.py > build_index
32  def chunk_text(
45
46      if chunk:
47          chunks.append(
48              {
49                  "chunk_id": chunk_id,
50                  "text": chunk,
51              }
52          )
53
54          chunk_id += 1
55
56      start += chunk_size - chunk_overlap
57
58  return chunks
59
```

```
rag.py 1, U X
code > rag.py > ...
75 def build_index():
119     embedding = embed_text(
120         chunk_text_value
121     )
122
123     collection.add(
124         ids=[vector_id],
125         documents=[chunk_text_value],
126         embeddings=[embedding],
127         metadatas=[
128             {
129                 "source": source,
130                 "chunk_id": chunk_id,
131             }
132         ],
133     )
134
135     total_chunks += 1
```

```
rag.py 1, U X
code > rag.py > build_index
75 def build_index():
85     collection = client.create_collection(
86         name="hw4_rag"
87     )
```

```
[ ]
● (base) optim@Timothys-MacBook-Pro code % python rag.py
lecture1.pdf: 28 chunks
lecture2.pdf: 31 chunks
lecture3.pdf: 30 chunks
lecture4.pdf: 25 chunks
lecture5.pdf: 29 chunks
lecture6.pdf: 36 chunks

Documents indexed: 6
Total chunks indexed: 179
Chunk size: 500
Chunk overlap: 50
Embedding model: nomic-embed-text
Vector store: Chroma
Collection count: 179
○ (base) optim@Timothys-MacBook-Pro code %
```

2. Retrieval: for each question, retrieve the top-k chunks (start with `top_k = 3`) and print the retrieved chunks with source and score before calling the LLM, so a retrieval problem is visible separately from a generation problem.

The retrieval step returned the top-3 chunks with source metadata, chunk IDs, and Chroma distance scores before any LLM generation call, allowing retrieval quality to be evaluated separately from generation quality.

```
rag.py 1, U X
code > rag.py > read_pdf
10 TOP_K = 3
11
```

```

rag.py 1, U X
code > rag.py > ...
160 def retrieve_chunks(
169
170     results = collection.query(
171         query_embeddings=[
172             question_embedding
173         ],
174         n_results=top_k,
175         include=[
176             "documents",
177             "metadatas",
178             "distances",
179         ],
180     )
181
182     print()
183     print(f"Question: {question}")
184     print(f"Top-k: {top_k}")
185     print()
186
187     retrieved = []
188
189     for i in range(top_k):
190         document = (
191             results["documents"][0][i]
192         )
193
194         metadata = (
195             results["metadatas"][0][i]
196         )
197
198         distance = (
199             results["distances"][0][i]
200         )
201
202         source = metadata["source"]
203         chunk_id = metadata["chunk_id"]
204
205         print(f"Result {i + 1}")
206         print(f"Source: {source}")
207         print(f"Chunk ID: {chunk_id}")
208         print(
209             f"Score/Distance: "
210             f"{distance:.4f}"
211         )
212         print("Chunk:")
213         print(document)
214         print("-" * 80)
215
216         retrieved.append(
217             {
218                 "rank": i + 1,
219                 "source": source,
220                 "chunk_id": chunk_id,
221                 "distance": distance,
222                 "text": document,
223             }
224         )
225
226     return retrieved
227

```

```

Problems 1 Output Debug Console Terminal Ports
• (base) optim@Timothys-MacBook-Pro code % cd /Users/optim/Desktop/260/data260-9981/code
python rag.py

Question: What is gradient descent?
Top-k: 3

Result 1
Source: lecture4.pdf
Chunk ID: 2
Score/Distance: 0.4096
Chunk:
function J
Gradient Descent
Wj
t := Wj
t-1 - α ∂J
∂W
Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
w
J(w)
Wj
Gradient descent method
• To find the weights that minimize a cost function,
gradient descent is commonly used as an optimization
algorithm.
• Gradient(vector of partial derivatives) gives the input
direction in which the function decreases
• Step0: pick a random starting point
• Step1: compute the gradient
• Step2: take a small step in the direction of the gradien

Result 2
Source: lecture6.pdf
Chunk ID: 20
Score/Distance: 0.5232
Chunk:
r method: based on Newton's method, L-BFGS → not cover
Source: An overview of gradient descent optimization algorithms
Comparison of optimizer functions
iteration
Accuracy
Source: Evolutionary Deep Learning, Micheal Lanham
Momentum update
Gradient descent update
x = x - learning rate*dx
Momentum update
v = β*v - learning rate*dx
x = x + v
v : velocity
β : momentum

Gradient descent with/wo Momentum
Source: Math and Architectures of Deep Learning, Krishnedu Chauhury
Adagrad
v = v + dx**2
x = x

Result 3
Source: lecture6.pdf
Chunk ID: 14
Score/Distance: 0.5370
Chunk:
decay
• Optimizer

Gradient Descent
Wj
t := Wj
t-1 - α ∂J
∂W
Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
Batch & Stochastic gradient descent
Source: Machine Learning, Emily Fox & Carlos Guestrin
Batch
Gradient
descent
Stochastic
Gradient
descent
∂J
∂W = - 1
m[]
i=1
m
[ [yi - yi xj
i]
∂J
∂W = - [yi - yi xj
i
(Sum over data points)
(Each time, pick different data point)
Mini-batch Gradient Descent
• Batch Gradient Descent
Computes the gradients based on full training set

• (base) optim@Timothys-MacBook-Pro code %

```

3. Run the same questions through three configurations:
(A) No RAG - question straight to the LLM, as a baseline.

```
def answer_no_rag(question):  
    prompt = f"""  
    Answer the following question.  
  
    Question:  
    {question}  
    """  
  
    answer = call_llm(prompt)  
  
    print()  
    print("=== CONFIGURATION A: NO RAG ===")  
    print(answer)  
  
    return answer
```

=== CONFIGURATION A: NO RAG ===

Gradient descent is an optimization algorithm used to minimize a function by iteratively adjusting its parameters in the direction of the steepest descent, which is the negative of the gradient of the function. Here's a structured explanation:

- Core Concept**: Gradient descent works by computing the gradient (vector of partial derivatives) of the loss function at a given parameter set. The algorithm updates the parameters in the opposite direction of the gradient to reduce the function's value, moving toward a minimum.
- Key Components**:
 - Learning Rate**: A hyperparameter that controls the step size during updates. Too large, and the algorithm may overshoot the minimum; too small, and convergence is slow.
 - Iterative Process**: Updates are performed repeatedly until the loss function stabilizes or a predefined number of iterations is reached.
- Variants**:
 - Batch Gradient Descent**: Uses the entire dataset to compute the gradient (stable but computationally heavy for large datasets).
 - Stochastic Gradient Descent (SGD)**: Uses a single data point per update (faster but noisier).
 - Mini-Batch Gradient Descent**: Balances computation and stability by using small subsets of data.
- Applications**: Widely used in training machine learning models (e.g., neural networks) to minimize loss functions. Parameters (weights) are adjusted iteratively to improve model performance.
- Challenges & Enhancements**:
 - Local Minima/Saddle Points**: The algorithm may converge to suboptimal solutions. Techniques like momentum or adaptive methods (e.g., Adam) help navigate these issues.
 - Learning Rate Scheduling**: Adjusting the learning rate over time (e.g., decay) improves convergence.

In summary, gradient descent is a foundational technique in machine learning, enabling models to learn by iteratively minimizing error through gradient-based updates.

Question: What is gradient descent?

Top-k: 3

Result 1

Source: lecture4.pdf

Chunk ID: 2

Score/Distance: 0.4096

Chunk:

function J

Gradient Descent

W

$r := W$

$r-1 - \alpha \cdot J$

αW

Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow

W

J(W)

W

Gradient descent method

- To find the weights that minimize a cost function, gradient descent is commonly used as an optimization algorithm.
- Gradient (vector of partial derivatives) gives the input direction in which the function decreases
- Step0: pick a random starting point
- Step1: compute the gradient
- Step2: take a small step in the direction of the gradient

Result 2

Source: lecture6.pdf

Chunk ID: 20

Score/Distance: 0.5232

Chunk:

r method: based on Newton's method, L-BFGS - not cover

Source: An overview of gradient descent optimization algorithms

Comparison of optimizer functions

iteration

Accuracy

Source: Evolutionary Deep Learning, Micheal Lanham

Momentum update

Gradient descent update

$x = x - \text{learning rate} \cdot dx$

Momentum update

$v = \beta v - \text{learning rate} \cdot dx$

$x = x + v$

v : velocity

β : momentum

Gradient descent with/wo Momentum

Source: Math and Architectures of Deep Learning, Krishnendu Chauhury

Adagrad

$v = v + dx \cdot dx^2$

$x = x$

Result 3

Source: lecture6.pdf

Chunk ID: 14

Score/Distance: 0.5370

Chunk:

decay

Optimizer

Gradient Descent

W

$r := W$

$r-1 - \alpha \cdot J$

αW

Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow

Batch & Stochastic gradient descent

Source: Machine Learning, Emily Fox & Carlos Guestrin

Batch

Gradient

descent

Stochastic

Gradient

descent

αJ

$\alpha W = -1$

$m[0]$

$i-1$

m

$[\sum y_i - y_i \cdot x_j]$

$i]$

αJ

$\alpha W = - \sum y_i - y_i \cdot x_j$

i

(Sum over data points)

(Each time, pick different data point)

Mini-batch Gradient Descent

Batch Gradient Descent

Computes the gradients based on full training set

(B) Basic RAG - top-3 raw chunks in the prompt.

```

262
263
264 def answer_basic_rag(
265     question,
266     retrieved,
267 ):
268     context = "\n\n".join(
269         chunk["text"]
270         for chunk in retrieved
271     )
272
273     prompt = f"""
274 Use the following retrieved context to answer the question.
275
276 Context:
277 {context}
278
279 Question:
280 {question}
281 """
282
283     answer = call_llm(prompt)
284
285     print()
286     print("=== CONFIGURATION B: BASIC RAG ===")
287     print(answer)
288
289     return answer

```

=== CONFIGURATION B: BASIC RAG ===

Gradient descent is an optimization algorithm used to minimize a cost function by iteratively adjusting the parameters (weights) of a model. It works by computing the gradient (vector of partial derivatives) of the cost function with respect to the parameters and updating them in the opposite direction of the gradient, scaled by a learning rate (α). This process moves the parameters toward a minimum of the function.

Key Steps:

1. **Initialization**: Start with random initial weights.
2. **Gradient Calculation**: Compute the gradient of the cost function J with respect to the parameters W .
3. **Parameter Update**: Adjust the weights using the formula:

$$W_j^{t+1} = W_j^t - \alpha \frac{\partial J}{\partial W_j}$$

Here, α controls the step size (learning rate), and the negative gradient direction ensures the function decreases.

Variants:

- **Batch Gradient Descent**: Uses the full training dataset to compute the gradient.
- **Stochastic Gradient Descent (SGD)**: Uses a single data point (or a small random subset) for each update.
- **Mini-batch Gradient Descent**: A balance between batch and SGD, using small subsets of data.

Gradient descent is foundational in training machine learning models, enabling them to learn optimal parameters by iteratively reducing prediction errors.

(C) Context-engineered RAG - drop irrelevant and duplicate chunks, order and label the survivors with their sources, and add grounding rules: answer only from the context, cite the source number, and refuse with "I cannot answer this question from the provided documents" when the evidence is insufficient.

```
def answer_context_rag(
    question,
    retrieved,
):
    survivors = prepare_context_engineered_chunks(
        retrieved
    )

    if not survivors:
        context = {
            "No relevant context was retrieved."
        }
    else:
        context_parts = []

        for i, chunk in enumerate(
            survivors,
            start=1,
        ):
            context_parts.append(
                f"[Source {i}] "
                f"{chunk['source']} "
                f"{chunk['chunk_id']}\n"
                f"{chunk['text']}"
            )

        context = "\n\n".join(
            context_parts
        )

    prompt = f"""
You are a grounded question-answering system.

Rules:
1. Answer only from the provided context.
2. Do not use outside knowledge.
3. Cite supporting evidence using [Source 1], [Source 2], etc.
4. If the provided context does not contain enough evidence to answer the question, respond exactly:
I cannot answer this question from the provided documents

Context:
{context}

Question:
{question}
"""

    answer = call_llm(prompt)

    print()
    print(
        "=== CONFIGURATION C: "
        "CONTEXT-ENGINEERED RAG ==="
    )
    print(answer)

    return answer
```

```

== CONFIGURATION C: CONTEXT-ENGINEERED RAG ==
Gradient descent is an optimization algorithm used to find the weights that minimize a cost function by iteratively adjusting parameters in the direction of the steepest descent, determined by the gradient of the function. The update rule for weights is given by:

$$W_j^{t+1} := W_j^t - \alpha \frac{\partial J}{\partial W_j}$$

where  $\alpha$  is the learning rate, and  $\frac{\partial J}{\partial W_j}$  represents the partial derivative of the cost function  $J$  with respect to the weights  $W_j$  [Source 1].

The method involves:
1. Selecting a random starting point.
2. Computing the gradient (vector of partial derivatives) to determine the direction of descent.
3. Taking a small step in that direction to update the parameters [Source 1].

This approach is foundational for training machine learning models by minimizing errors in predictions.
(base) opting@timothys-MacBook-Pro code %

```

```

rag.py 1, u X
code > rag.py > answer_context_rag
292 def prepare_context_engineered_chunks(
293     retrieved,
294     max_distance=0.65,
295 ):
296     survivors = []
297     seen_text = set()
298
299     for chunk in retrieved:
300         text = chunk["text"].strip()
301
302         if chunk["distance"] > max_distance:
303             continue
304
305         if text in seen_text:
306             continue
307
308         seen_text.add(text)
309         survivors.append(chunk)
310
311     return survivors
312

```

```

rag.py 1, u X
code > rag.py > answer_context_rag
314 def answer_context_rag(
345     You are a grounded question-answering system.
346
347     Rules:
348     1. Answer only from the provided context.
349     2. Do not use outside knowledge.
350     3. Cite supporting evidence using [Source 1], [Source 2], etc.
351     4. If the provided context does not contain enough evidence to answer the question, respond exactly:
352     I cannot answer this question from the provided documents
353

```

4. Test set of six questions: o Q1: answer in one chunk o Q2: answer needs two chunks o Q3: similar information across documents o Q4: ambiguous o Q5: answer not in the documents o Q6: unrelated Q5 and Q6 must be refused, not answered.

Question	Purpose
Q1: What is gradient descent?	Answer is available in one retrieved chunk
Q2: According to the lectures, how does batch gradient descent compute its gradient using the training data, and what velocity and parameter update equations are used by momentum gradient descent?	Answer requires combining two retrieved chunks
Q3: How is gradient descent described across the lecture documents?	Similar information appears across multiple documents/chunks
Q4: What does momentum mean?	Ambiguous question
Q5: What is SJSU's address?	Answer is not present in the document collection
Q6: What is today's weather?	Unrelated to the document collection

The six-question test set was designed to evaluate different retrieval and grounding conditions.

Q1 tests a question that can be answered from one chunk.

Q2 requires evidence from two chunks: one describing batch gradient descent using the full training set and another providing the momentum update equations.

Q3 tests similar information distributed across the lecture documents.

Q4 is intentionally ambiguous.

Q5 asks for information that is not contained in the corpus.

Q6 is unrelated to the corpus.

Each question was tested with No RAG, Basic RAG, and Context-Engineered RAG. Q5 and Q6 were required to be refused by the context-engineered system.

Question 1

What is gradient descent?

Top-k: 3

Result 1

Source: lecture4.pdf

Chunk ID: 2

Score/Distance: 0.4096

Gradient descent method

- To find the weights that minimize a cost function, gradient descent is commonly used as an optimization algorithm.
- Gradient gives the direction in which the function decreases.
- Step 0: pick a random starting point
- Step 1: compute the gradient
- Step 2: take a small step in the direction of the gradient

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

Gradient descent is an optimization algorithm used to find the weights that minimize a cost function by iteratively adjusting parameters in the direction of the negative gradient.

The weight update rule is:

$$W_j^t := W_j^{(t-1)} - \alpha \partial J / \partial W$$

[Source 1]

Question 2:

According to the lectures, how does batch gradient descent compute its gradient using the training data, and what velocity and parameter update equations are used by momentum gradient descent?

Result 1

Source: lecture6.pdf

Chunk ID: 15

Score/Distance: 0.3700

Computes the gradients based on full training set

...

"batch gradient descent" calculates the gradient using the entire training dataset for each parameter update

Result 2

Source: lecture6.pdf

Chunk ID: 20

Score/Distance: 0.4346

Momentum update

Gradient descent update

$x := x - \text{learning rate} * dx$

Momentum update

$v = \beta * v - \text{learning rate} * dx$

$x := x + v$

v : velocity

β : momentum

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

Batch gradient descent computes its gradient using the entire training dataset for each parameter update, as it calculates the gradient based on the full training set [Source 1].

Momentum gradient descent uses the following equations:

- Velocity update:

$v = \beta * v - \text{learning rate} * \partial J / \partial x$ [Source 2]

- Parameter update:

$x := x + v$ [Source 2]

Question 3: How is gradient descent described across the lecture documents?

Result 1

Source: lecture4.pdf

Chunk ID: 2

Score/Distance: 0.5412

Gradient descent method

- Used to minimize a cost function
- Compute the gradient
- Take a step in the descent direction

Result 2

Source: lecture6.pdf

Chunk ID: 20

Score/Distance: 0.5719

Momentum update

Gradient descent update

$x := x - \text{learning rate} * dx$

$v = \beta * v - \text{learning rate} * dx$

$x := x + v$

Result 3

Source: lecture6.pdf

Chunk ID: 15

Score/Distance: 0.6051

Batch Gradient Descent

Computes gradients using the entire training dataset

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

Gradient descent is described across the lecture documents as an optimization algorithm used to minimize a cost function by iteratively adjusting weights in the direction of the steepest descent.

[Source 1] describes standard gradient descent.

[Source 2] describes momentum and adaptive methods.

[Source 3] describes batch, stochastic, and mini-batch gradient descent.

Question 4: What does momentum mean?

Top-k: 3

Result 1

Source: lecture3.pdf

Chunk ID: 25

Score/Distance: 0.9016

Entropy in information ...

Result 2

Source: lecture3.pdf

Chunk ID: 24

Score/Distance: 0.9696

Entropy in information ...

Result 3

Source: lecture3.pdf

Chunk ID: 23

Score/Distance: 0.9768

Classification accuracy / entropy ...

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

I cannot answer this question from the provided documents

Question 5: What is SJSU's address?

Result 1

Source: lecture1.pdf

Chunk ID: 0

Score/Distance: 0.8547

San Jose State University

Instructor information

Office: ISB 866

Course information

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

I cannot answer this question from the provided documents

Question 6: What is today's weather?

Result 1

Source: lecture4.pdf

Chunk ID: 0

Score/Distance: 0.9598

DATA 255

Deep Learning and Computer Vision

Today's Topics:

- Model optimization
- Backward propagation

Result 2

Source: lecture3.pdf

Chunk ID: 23

Score/Distance: 1.0365

Classification accuracy / entropy

Result 3

Source: lecture1.pdf

Chunk ID: 0

Score/Distance: 1.0379

DATA 255

San Jose State University

Course information

=== CONFIGURATION C: CONTEXT-ENGINEERED RAG ===

I cannot answer this question from the provided documents

The results show that the test set exercised the intended retrieval conditions.

Q1 was answered from directly relevant evidence in a single chunk.

Q2 required combining the batch-gradient-descent description with a separate momentum-update chunk.

Q3 synthesized related gradient-descent information from multiple retrieved sources.

For the ambiguous Q4, retrieval returned unrelated evidence and the context-engineered configuration refused to answer.

Q5 and Q6 were also refused because the retrieved context did not provide sufficient supporting information.

5. Sweep the context size: run at least one question at $k = 1$, 3 , and 5 , and report whether more context helped, when irrelevant chunks appeared, and which k was best - more context is not always better context.

To study the effect of context size, I evaluated Q3, “How is gradient descent described across the lecture documents?”, using $k = 1$, $k = 3$, and $k = 5$.

k	Observation
1	Retrieved only the core gradient-descent chunk. The answer was correct but limited to the basic definition, update rule, and procedure.
3	Added relevant chunks covering momentum and batch/stochastic/mini-batch gradient descent. The answer became more complete while remaining focused.
5	Added two more chunks about PyTorch Autograd and derivatives. These were related to gradient computation, but they were less directly relevant to the question and added background and implementation details

More context helped when increasing from $k = 1$ to $k = 3$, because the additional chunks contributed useful information about gradient-descent variants and momentum. However, increasing to $k = 5$ introduced less directly relevant context about automatic differentiation and derivatives. The answer remained correct, but the extra context did not substantially improve the explanation of gradient descent. Therefore, $k = 3$ was the best context size for this question, because it provided the best balance between coverage and relevance. This experiment demonstrates that more context is not always better context: after the most relevant chunks have been retrieved, increasing k can introduce redundant or less relevant information.

6. Evaluate: for each question and configuration, record correct retrieval, correct answer, ground (answer supported by the cited chunks), and refused-when-needed; summarize accuracy, faithfulness, format compliance, and robustness in one table.

Correct retrieval indicates whether the retrieved chunks contained the evidence needed for the question. Grounded indicates whether the generated answer was supported by the retrieved context. Retrieval and grounding are marked N/A for No RAG because that configuration does not retrieve documents. Refused-when-needed applies specifically to Q5 and Q6.

Question	Configuration	Correct Retrieval	Correct Answer	Grounded	Refused When Needed
Q1	No RAG	N/A	Yes	N/A	N/A
Q1	Basic RAG	Yes	Yes	Yes	N/A
Q1	Context-Engineered RAG	Yes	Yes	Yes	N/A
Q2	No RAG	N/A	Yes	N/A	N/A
Q2	Basic RAG	Yes	No	Partial	N/A
Q2	Context-Engineered RAG	Yes	Yes	Yes	N/A
Q3	No RAG	N/A	Yes	N/A	N/A
Q3	Basic RAG	Yes	Yes	Yes	N/A
Q3	Context-Engineered RAG	Yes	Yes	Yes	N/A
Q4	No RAG	N/A	No	N/A	N/A
Q4	Basic RAG	No	Yes	Yes	N/A
Q4	Context-Engineered RAG	No	Yes	Yes	N/A
Q5	No RAG	N/A	No	N/A	No
Q5	Basic RAG	N/A	Yes	Yes	Yes
Q5	Context-Engineered RAG	N/A	Yes	Yes	Yes
Q6	No RAG	N/A	Yes	N/A	Yes
Q6	Basic RAG	N/A	Yes	Yes	Yes
Q6	Context-Engineered RAG	N/A	Yes	Yes	Yes

Configura tion	Acc ura cy	Faithfulness	Format Compliance	Robustness
No RAG	4/6 (66. 7%)	N/A; no retrieved grounding	No citation/grounding format	Weak; failed to refuse Q5
Basic RAG	5/6 (83. 3%)	Mostly faithful; Q2 equation inconsistency	No enforced source citations	Good; weaker on multi-chunk synthesis
Context-E ngineered RAG	6/6 (10 0%)	Fully supported by selected context	Source citations and required refusals	Strong across all six test conditions

7. Write a short analysis (300–500 words): which questions retrieved the right chunks, which context changes helped (top_k, de-duplication, chunk size, source metadata, ordering, shorter context, grounding rules), whether the model hallucinated on Q5/Q6, and how retrieval quality, context quality, and the prompt together shaped answer quality.

The retrieval results varied depending on the type of question. Q1 retrieved a highly relevant chunk from lecture4.pdf that directly described gradient descent, its update rule, and its basic procedure. Q2 also retrieved the correct evidence, but the answer required combining two chunks: one from lecture6.pdf explaining that batch gradient descent computes gradients using the full training set, and another containing the momentum velocity and parameter update equations. Q3 retrieved several relevant chunks describing standard gradient descent, momentum, and batch/stochastic variants, which allowed the system to synthesize information across the lecture documents. In contrast, Q4 was ambiguous, and retrieval returned unrelated entropy chunks. Q5 and Q6 also retrieved irrelevant or insufficient evidence because the requested information was not available in the corpus.

Several context-engineering choices improved answer quality. The initial chunk size of 500 characters with 50-character overlap produced reasonably focused pieces of evidence while preserving some continuity between neighboring chunks. Source metadata and chunk IDs made retrieved evidence traceable, while ordering chunks by retrieval score placed the most relevant evidence first. The context-engineered configuration also removed low-quality chunks using a distance threshold and avoided duplicate text before constructing the prompt. Most importantly, the grounding rules required the model to answer only from the supplied context, cite labeled sources, and refuse when the available evidence was insufficient.

The context-size sweep demonstrated that more context was not always better. With $k = 1$, Q3 produced a correct but narrow answer based only on the core gradient-descent chunk. Increasing to $k = 3$ added useful information about momentum and batch, stochastic, and mini-batch gradient descent. At $k = 5$, additional chunks about PyTorch Autograd and derivatives appeared. These chunks were related to gradient computation but were less directly relevant, so $k = 3$ provided the best balance between coverage and relevance.

The No-RAG model hallucinated on Q5 by supplying SJSU's address even though that information was not available in the document collection. The context-engineered system instead refused both Q5 and Q6 using the required refusal message. Overall, answer quality depended on all three stages working together: retrieval needed to find useful chunks, context engineering needed to select and organize those chunks, and the prompt needed to enforce grounding and refusal behavior. Good retrieval alone was not sufficient when the prompt allowed unsupported synthesis, while stronger context selection and grounding rules produced more faithful and robust answers.

AI USE

1. What I used an AI assistant for and what I did myself

I used an AI assistant to help interpret the homework requirements, review my implementation, suggest code structure, identify possible mistakes, improve grammar and formatting, and help organize the written report. I also used the AI assistant to review code snippets for syntax and structure, clarify assignment expectations, and help present tables, explanations, and experimental results more clearly.

For Part 4, I used the AI assistant to help design the six-question RAG test set, review the retrieval and context-engineering logic, suggest the $k = 1, 3$, and 5 context-size sweep, and help summarize the experimental results.

I personally implemented and ran the React frontend, FastAPI backend, MySQL database, authentication and session handling, CRUD endpoints, naive N+1 and fixed endpoints, benchmarking, indexing, RAG pipeline, Chroma vector store, Ollama models, retrieval experiments, and k-sweep. I also executed the commands locally, collected the actual outputs, inspected the results, verified that the system behaved as expected, and prepared the final report.

2. One AI-produced output that was wrong or unsuitable, or one thing I independently verified

One AI-assisted suggestion initially used a Q2 wording that was intended to require two retrieved chunks. However, after testing it, I found that the first version could actually be answered from a single retrieved chunk because that chunk contained both standard gradient-descent and momentum information.

3. How I detected the problem or verified the result

I detected the issue by running the retrieval pipeline and inspecting the printed top-k chunks before the LLM response. The output showed that the highest-ranked chunk already contained both pieces of information needed to answer the original Q2. Therefore, the question did not satisfy the assignment requirement that Q2 require information from two separate chunks.

4. What I changed and why it works now

I rewrote Q2 so that it asks how batch gradient descent computes its gradient using the training data and also asks for the velocity and parameter-update equations used by momentum gradient descent.

After rerunning the retrieval, one chunk contained the batch-gradient-descent information, while a separate chunk contained the momentum equations. The Context-Engineered RAG answer then combined the two pieces of evidence and cited both retrieved sources. Therefore, the revised Q2 now correctly tests multi-chunk retrieval and synthesis.